FastMCP Observability Callback Skills Project Guide

Reorganized Architecture, Observability Layer, and File-by-File Developer Notes

Scope: This guide explains the reorganized project structure we discussed, including the updated JSONL request record with request_summary and individual tool_calls. It is a developer guide, not a full source-code listing.

Main idea: The app keeps request handling, session management, agent execution, MCP runtime, models, and observability separated while preserving the same runtime behavior.

How to read this guide

- Start with the architecture flow to understand how one /chat request moves through the system.
- Use the file-by-file sections when you want to know where a class, function, span, or JSON record is created.
- Use the observability section to understand what belongs in spans versus what belongs in JSONL metrics.

Final project layout

```
fastmcp_observability_callback_simple/
├── backend/
│   ├── main.py
│   ├── config.py
│   ├── agent/
│   │   └── engine.py
│   ├── mcp/
│   │   └── runtime.py
│   ├── sessions/
│   │   └── store.py
│   ├── models/
│   │   ├── api.py
│   │   ├── agent.py
│   │   ├── session.py
│   │   └── observability.py
│   └── observability/
│       ├── metrics.py
│       ├── otel.py
│       └── request_observer.py
├── gateways/
│   └── core_gateway.py
├── mcp_server/
│   ├── calculator_server.py
│   └── quote_counter_server.py
├── skills/
│   ├── math/SKILL.md
│   └── quote_count/SKILL.md
├── README.md
├── pyproject.toml
└── .env.example
```

Area	Responsibility
backend/main.py	FastAPI app entry point and request route wiring.
backend/config.py	Typed runtime settings loaded from .env and environment variables.
backend/agent/engine.py	Creates the Deep Agent and runs one chat turn.
backend/mcp/runtime.py	Connects to MCP gateways and loads visible tools.
backend/sessions/store.py	Stores per-session conversation state and locking.
backend/models/*.py	Typed data contracts grouped by layer.
backend/observability/*.py	OpenTelemetry setup, callback-based observation, and JSONL writing.
gateways/core_gateway.py	Mounts built-in MCP servers under namespaces.
mcp_server/*.py	Standalone MCP tool servers.
skills/*/SKILL.md	Native Deep Agents skill instructions loaded from disk.

Request flow

```
POST /chat
  -> backend/main.py
      validates ChatRequest
      starts request timer
      calls ChatSessionStore.chat(...)

  -> backend/sessions/store.py
      finds or creates ChatSession
      waits for the per-session lock
      calls AgentEngine.respond(...)

  -> backend/agent/engine.py
      builds agent_input from prior messages + user message
      attaches RequestObserver callbacks
      calls Deep Agent .ainvoke(...)
      returns EngineResult

  -> backend/observability/request_observer.py
      counts real LLM starts
      records real tool starts/ends/errors
      creates per-tool spans and tool-call records

  -> backend/main.py
      writes one JSONL telemetry row
      returns ChatResponse
```

Key separation: The API route does not directly know how the agent works. The session layer does not directly know how tools work. The observer records what actually happened through LangChain callback events.

Observability design

Trace spans versus JSONL metrics

Signal type	Used for	Examples
Trace spans	Interactive debugging and timing relationships.	request span, session.chat, engine.respond, tool.invoke add

JSONL metrics	Durable per-request records for later analysis.	request_summary, tool_calls list, timings, success/error info
API response metrics	Immediate metrics returned to the caller.	total_response_time_ms, llm_calls, tool_calls

- Request span should keep high-level request success and total elapsed time.
- Session span should focus on session state and lock wait time.
- Agent span should focus on agent timing, LLM count, and aggregate tool metrics.
- Tool spans should focus on individual tool execution details.
- JSONL should be richer than spans because it is your durable analysis record.

Recommended JSONL record shape

```
{
  "request_summary": {
    "mode": "deep_agent",
    "session_id": "demo",
    "trace_id": "abc123",
    "success": true,
    "error": null,
    "user_message_len": 12,
    "total_response_time_ms": 1128.44,
    "engine_time_ms": 1094.72,
    "lock_wait_ms": 0.02,
    "llm_calls": 2,
    "tool_call_count": 2,
    "tool_success_count": 2,
    "tool_error_count": 0,
    "tool_total_duration_ms": 55.9,
    "tool_max_duration_ms": 31.2,
    "unique_tools_used": ["math_add", "math_subtract"],
    "ts": "2026-04-30T22:30:00.000000+00:00"
  },
  "tool_calls": [
    {
      "tool_call_number": 1,
      "name": "math_add",
      "success": true,
      "duration_ms": 24.7,
      "input_size": 16,
      "input_preview": "{"a":5,"b":6}",
      "response_size": 2,
      "response_preview": "11",
      "error_type": null,
      "error_message": null
    }
  ]
}
```

Preview fields: input_preview and response_preview intentionally store compact previews, not full payloads. This keeps metrics useful without flooding logs or accidentally storing large/sensitive values.

File-by-file guide

backend/main.py

Purpose: FastAPI entry point. Creates the app, wires runtime dependencies, exposes health and chat routes, and writes one telemetry record per request.

Name	Type	What it does
create_app(settings=None)	function	Builds the FastAPI app. Loads settings, configures OpenTelemetry, defines lifespan startup/shutdown, adds CORS, and registers routes.
lifespan(app)	nested async context manager	Runs during app startup and shutdown. Starts MCPRuntime, builds AgentEngine, creates ChatSessionStore and JSONLMetricsWriter, and stops MCP runtime on shutdown.
health()	route handler	Simple GET /health endpoint that returns {"status":"ok"}.
chat(req)	route handler	POST /chat endpoint. Measures total request time, calls the session store, writes ChatTelemetryEvent to JSONL, and returns ChatResponse.
app = create_app()	module variable	Creates the ASGI app used by uvicorn backend.main:app.

backend/config.py

Purpose: Defines typed runtime configuration so the rest of the app does not work with raw environment strings.

Name	Type	What it does
_csv(value)	helper function	Converts a comma-separated environment variable into a list of non-empty strings. Used for CORS origins.
_bool(value, default=False)	helper function	Parses common truthy strings like 1, true, yes, and on into booleans.
Settings	Pydantic model	Typed configuration object for model settings, gateway URLs, CORS, metrics path, skills path, and OpenTelemetry settings.
Settings.from_env()	classmethod	Loads .env, reads environment variables, applies defaults, and returns a frozen Settings object.

backend/models/api.py

Purpose: API request and response contracts for the FastAPI layer.

Name	Type	What it does
ChatRequest	Pydantic model	Incoming /chat body. Requires session_id and message.
ChatResponseMetrics	Pydantic model	Metrics returned in the HTTP response, such as total time, engine time, LLM calls, tool calls, and lock wait.
ChatResponse	Pydantic model	Outgoing /chat response. Contains reply, tools_used, mode, trace_id, and metrics.

backend/models/agent.py

Purpose: Agent-layer data model returned by AgentEngine.respond().

Name	Type	What it does
EngineResult	Pydantic model	Internal return object with reply text, tools used, engine metrics, and updated conversation state.

backend/models/session.py

Purpose: Session-layer data models for storing and returning conversation state.

Name	Type	What it does
ChatSession	Pydantic model	Holds one session state dictionary and one asyncio.Lock to prevent concurrent writes to the same session history.
ChatResult	Pydantic model	Result returned from ChatSessionStore.chat() to the API layer: reply, tools_used, metrics, and trace_id.

backend/models/observability.py

Purpose: Observability data models used by callbacks, agent metrics, and JSONL telemetry.

Name	Type	What it does
ToolCallStarted	Pydantic model	Temporary in-memory record created when a tool starts. Stores index, name, run_id, input size/preview, and start time.
ToolCallRecord	Pydantic model	Completed tool-call record. Stores tool_call_number, name, success, duration, input/response preview fields, and error details.
ToolSummary	Pydantic model	Aggregate rollup of all tools used in one request.
ToolSummary.to_span_attributes()	method	Converts aggregate tool metrics into OpenTelemetry span attributes for

		the engine span.
EngineMetrics	Pydantic model	Agent-level metrics passed through the session layer and written to the JSONL record.
ChatTelemetryEvent	Pydantic model	Full telemetry event object assembled at the API layer after a request finishes.
ChatTelemetryEvent.to_jsonl_row()	method	Returns the final nested JSON shape: request_summary plus tool_calls.

backend/agent/engine.py

Purpose: Owns the shared Deep Agent and executes one chat turn with observability callbacks attached.

Name	Type	What it does
DEFAULT_SYSTEM_PROMPT	constant	Fallback system prompt used when APP_SYSTEM_PROMPT is not provided.
AgentEngine	class	Creates the Deep Agent at startup and runs user messages through it.
AgentEngine.__init__(settings, mcp_runtime)	method	Stores Settings and MCPRuntime. Initializes self.agent as None until start() runs.
AgentEngine.start()	async method	Loads MCP tools, records startup tool metadata on the engine.start span, and creates the Deep Agent with model, tools, system prompt, and skills directory.
AgentEngine.respond(session_id, state, user_text)	async method	Runs one chat turn. Creates engine.respond span, attaches RequestObserver callbacks, invokes the agent, extracts the reply, builds EngineMetrics, and returns EngineResult.
AgentEngine._last_ai_message_text(messages)	static method	Scans the final message list backward and returns the last assistant message text.

backend/mcp/runtime.py

Purpose: Loads tools from configured MCP gateways and keeps gateway sessions alive for the app lifetime.

Name	Type	What it does
MCPRuntime	class	Manages MCP client, connection config, active gateway sessions, and cached tools.
MCPRuntime.__init__(settings)	method	Stores settings and initializes empty runtime state.
MCPRuntime.start()	async method	Builds the MCP connection map for core and optional BYOT gateway, then creates MultiServerMCPClient.
MCPRuntime.get_all_tools()	async method	Loads tools from all gateways,

		deduplicates them by name, records mcp.get_all_tools span attributes, and returns the final tool list.
MCPRuntime._load_tools_for_gateway(alias)	async helper	Opens one gateway session, loads its tools, stores the session context, and caches the tools for that alias.
MCPRuntime._deduplicate_tools_by_name()	helper	Returns one tool per name while preserving load order across gateways.
MCPRuntime._join_tool_names(tools)	static helper	Builds a comma-separated tool-name string for span attributes.
MCPRuntime.stop()	async method	Closes active MCP sessions and clears runtime state during app shutdown.

backend/sessions/store.py

Purpose: Manages per-session conversation history and prevents concurrent updates to the same session.

Name	Type	What it does
ChatSessionStore	class	In-memory session manager that calls AgentEngine for each chat turn.
ChatSessionStore.__init__(engine)	method	Stores the engine, initializes a session dictionary, and creates a lock for session-map access.
ChatSessionStore.chat(session_id, message)	async method	Gets or creates the session, measures lock wait time, invokes the engine, updates session state, and returns ChatResult.
ChatSessionStore._get_or_create_session(session_id)	async helper	Safely returns an existing ChatSession or creates a new one under sessions_lock.

backend/observability/metrics.py

Purpose: Writes one JSON object per chat request into a JSONL file.

Name	Type	What it does
JSONLMetricsWriter	class	Small thread-safe writer for local JSONL telemetry.
JSONLMetricsWriter.__init__(path)	method	Creates parent directory if needed and prepares a threading.Lock for writes.
JSONLMetricsWriter.write(event)	method	Converts ChatTelemetryEvent to a JSONL row, adds UTC timestamp inside request_summary, and appends the row to disk.

backend/observability/otel.py

Purpose: Small OpenTelemetry setup and helper module.

Name	Type	What it does
_CONFIGURED	module variable	Prevents OpenTelemetry provider setup from running more than once per process.
setup_otel(enabled, service_name, otlp_endpoint)	function	If enabled, creates TracerProvider, sets service metadata, chooses OTLP exporter or console exporter, and registers the provider.
instrument_fastapi(app, enabled)	function	Adds automatic FastAPI request tracing when OpenTelemetry is enabled.
get_tracer(name)	function	Returns a tracer for the given module name.
current_trace_id()	function	Returns the active trace id as a 32-character hex string, or None if no valid span exists.

backend/observability/request_observer.py

Purpose: LangChain callback handler that records real LLM/tool activity during one agent request.

Name	Type	What it does
RequestObserver	class	Collects LLM call count, active tool state, completed tool records, and per-tool spans.
RequestObserver.__init__()	method	Initializes counters, active tool maps, completed records, and a set of seen LLM run IDs.
on_chat_model_start(...)	callback	Called by LangChain when a chat model starts. Delegates to _record_llm_start.
on_llm_start(...)	callback	Called by LangChain when a non-chat LLM starts. Delegates to _record_llm_start.
on_tool_start(...)	callback	Called when a tool starts. Creates ToolCallStarted, assigns a tool number, captures input preview, and starts a tool.invoke span.
on_tool_end(...)	callback	Called when a tool succeeds. Extracts the useful tool response, records duration and response preview, appends ToolCallRecord, and ends the tool span.
on_tool_error(...)	callback	Called when a tool fails. Records error type/message, appends failed ToolCallRecord, records exception on the span, and ends the span.
tools_used()	method	Returns tool names in execution order.
tool_summary()	method	Builds aggregate tool counts, durations, unique names, and success/error counts.

_record_llm_start(run_id, model_type)	helper	Deduplicates LLM run IDs and increments llm_calls. Also adds an llm.started event to the current span.
_extract_tool_response(output)	helper	Pulls the useful response from output.artifact.structured_content or output.content instead of stringifying the whole wrapper object.
_extract_content_text(content)	helper	Handles content strings or LangChain content lists and extracts text values.
_compact_json_or_text(value)	helper	Serializes structured values as compact JSON; falls back to str(value).
_duration_ms(start_time_s)	static helper	Converts elapsed perf_counter time into milliseconds.
_preview(value, max_chars=300)	static helper	Normalizes a value to one line and truncates it for JSONL previews.

gateways/core_gateway.py

Purpose: Creates the built-in FastMCP gateway that exposes internal tool servers under stable namespaces.

Name	Type	What it does
build_gateway()	function	Creates FastMCP("Core Gateway"), mounts calculator server under math namespace, mounts quote counter server under quote namespace, and returns the gateway.
if __name__ == "__main__"	script entry	Runs the gateway as an HTTP MCP server using host, port, and path environment variables.

mcp_server/calculator_server.py

Purpose: Standalone MCP server exposing simple calculator tools.

Name	Type	What it does
mcp = FastMCP("Calculator")	module object	Creates the MCP server instance.
add(a, b)	MCP tool	Adds two numbers and returns an int when the result is mathematically an integer.
subtract(a, b)	MCP tool	Subtracts b from a and returns an int when the result is mathematically an integer.
if __name__ == "__main__"	script entry	Runs the calculator MCP server over HTTP.

mcp_server/quote_counter_server.py

Purpose: Standalone MCP server for extracting and counting the first quoted text segment.

Name	Type	What it does

mcp = FastMCP("Quote Counter")	module object	Creates the MCP server instance.
_first_quoted_segment(text)	helper function	Uses regex to find the first double-quoted or single-quoted segment in text.
count_quoted_chars(text)	MCP tool	Returns a dict with quoted_text and quoted_char_count.
if __name__ == "__main__"	script entry	Runs the quote counter MCP server over HTTP.

skills/math/SKILL.md

Purpose: Native Deep Agents skill instructions for math-style requests.

Name	Type	What it does
Purpose	skill instruction	Tells the agent how and when to use calculator tools.
Behavior guidance	skill instruction	Encourages tool use for arithmetic or when exact calculation improves correctness.

skills/quote_count/SKILL.md

Purpose: Native Deep Agents skill instructions for quoted-text counting requests.

Name	Type	What it does
Purpose	skill instruction	Tells the agent how and when to use the quote-counting tool.
Behavior guidance	skill instruction	Encourages using the quote counter when the user asks about characters inside quotes.

What happens during startup

```
uvicorn backend.main:app
  -> create_app()
  -> lifespan startup
      -> Settings.from_env()
      -> setup_otel(...)
      -> MCPRuntime.start()
      -> MCPRuntime.get_all_tools()
      -> AgentEngine.start()
      -> create_deep_agent(..., skills=[settings.skills_dir])
```

- The MCP runtime is started once for the app lifetime.
- Tools are loaded once during AgentEngine.start().
- The Deep Agent is shared across requests.
- Per-request state lives in ChatSessionStore, not inside the FastAPI route.

What happens during one chat request

```
1. /chat receives ChatRequest.
2. main.py starts total request timer.
3. ChatSessionStore gets the session and waits for the per-session lock.
4. AgentEngine.respond creates RequestObserver and invokes the Deep Agent.
5. RequestObserver records real LLM and tool callback events.
6. AgentEngine returns EngineResult with reply, updated state, metrics, and tool_records.
7. Session store saves updated message state and attaches lock_wait_ms.
8. main.py writes ChatTelemetryEvent to JSONL.
9. main.py returns ChatResponse to the caller.
```

Span cleanup decisions

Span or event	Keep or remove	Reason
FastAPI request span	Keep	Useful top-level request timing and status.
session.chat span	Keep	Only place that clearly shows session lock wait and message count before/after.
engine.respond span	Keep	Best summary span for agent time, LLM calls, and aggregate tool metrics.
engine.agent_invoke span	Remove	Mostly duplicates engine.respond and adds trace noise.
tool.invoke <name> span	Keep	Best place for individual tool timing and success/error info.
tool.completed event on engine span	Remove	Duplicates individual tool.invoke spans.
llm.started event	Keep	Useful light event showing real callback-based LLM calls without extra spans.

Where to add future observability fields

Want to capture...	Best location	Why
A field returned in the HTTP response	backend/models/api.py and backend/main.py	API contract lives in api models; route builds response.
A durable JSONL request-level field	ChatTelemetryEvent.to_jsonl_row()	This controls the final JSONL shape.
A durable individual tool field	ToolCallRecord and RequestObserver.on_tool_end/on_tool_error	Tool-call records are created by callbacks.
An aggregate tool metric	ToolSummary and EngineMetrics	Summary flows through the agent/session/API layers.
A trace-only attribute	The span where that operation is measured	Keep spans focused: request, session, agent, or tool.

Important design rules

- Keep the FastAPI route thin: validate request, call session store, write telemetry, return response.
- Keep session behavior in sessions/store.py, especially locks and message state.
- Keep agent behavior in agent/engine.py, including prompt, tools, Deep Agent invocation, and reply extraction.
- Keep MCP loading in mcp/runtime.py, not in the route or agent callback code.
- Keep all Pydantic models in backend/models so ownership is clear by layer.
- Use RequestObserver for real LLM/tool events; do not estimate tool usage from messages.
- Use spans for debugging and JSONL for durable analysis.
- Prefer previews over full raw input/output values in telemetry records.

Quick troubleshooting map

Symptom	First file to inspect	Likely area
/chat route fails before agent runs	backend/main.py	Request handling, app state, settings, CORS, telemetry write.
Agent cannot see tools	backend/mcp/runtime.py	Gateway URL, MCP session, load_mcp_tools, deduplication.
Tool server not reachable	gateways/core_gateway.py or mcp_server/*.py	MCP server host/port/path or gateway mount URL.
Wrong llm_calls or tool_calls	backend/observability/request_observer.py	LangChain callback flow.
JSONL shape is wrong	backend/models/observability.py	ChatTelemetryEvent.to_jsonl_row().
Trace id missing	backend/observability/otel.py	OpenTelemetry enabled/configured and active span context.
Concurrent requests corrupt session history	backend/sessions/store.py	Session lock behavior.

Appendix: old-to-new file mapping

Old file	New file
backend/schemas.py	backend/models/api.py
backend/observability/models.py	backend/models/observability.py
EngineResult inside backend/agent.py	backend/models/agent.py
ChatSession and ChatResult inside backend/session_store.py	backend/models/session.py
backend/agent.py	backend/agent/engine.py
backend/mcp_runtime.py	backend/mcp/runtime.py
backend/session_store.py	backend/sessions/store.py

Final note: The refactor is intentionally a folder clarity refactor. The app behavior stays the same: a request enters FastAPI, flows through session state, invokes the Deep Agent, uses MCP tools when appropriate, records observability, and returns a response.